PRÁCTICA DE LABORATORIO / TALLER Nro. 3

Carrera de Computación

A. DATOS INFORMATIVOS		
Asignatura: Algoritmos, Análisis y Programación Paralela	**Ciclo / Semestre:** Séptimo	**Paralelo:** A
Docente: Roberth Gustavo Figueroa Díaz	**Período Académico:** Septiembre 2025 – Febrero 2026	
Alumnos: Gerardo Israel Herrera Campoverde Bayron Alejandro Jimenez Cango Alexis Gradyn Ludeña Cueva Luis Alberto Medina Chamba Nayely Cruzcaya Ramirez Herrera.		

B. INFORMACIÓN GENERAL	
Unidad: 3.Patrones algorítmicos en paralelos y análisis del rendimiento.	
Tema: Patrones Algorítmicos en Paralelo y Análisis del Rendimiento usando GPU	
Fecha: Loja, 01 de Febrero de 2026	**Nro. horas:** 3h
Objetivos:	
● Analizar el rendimiento de algoritmos paralelos usando GPU con CUDA.	
Corresponde al resultado de aprendizaje:	
Identifica tareas independientes en un programa que podría ser paralelizado, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.	
Recursos y/o materiales:	
● Computador. – Computadora con acceso a internet. ● Navegador. – Revisión de información por parte del estudiante en la web. ● Bibliografía. – Textos físicos, virtuales y materiales facilitados en el Sílabo. ● Biblioteca virtual. – Acceso a recursos digitales como libros y artículos. ● EVA . – Entorno virtual de aprendizaje de la UNL.	

C. DESARROLLO

Instrucciones:

1. Revisar el contenido facilitado por el profesor: diapositivas, libro base, sílabo de la asignatura, recursos digitales, use el libro base e información complementaria; así como, internet para reforzar los contenidos estudiados.
2. Revise detalladamente el material indicado en clase.
3. Realizar la lectura del texto base y complementarios sobre las temáticas abordadas.
4. Para el desarrollo de los ejercicios puede usarse las herramientas de su interés tales como: lenguaje Python, Java o C++, herramienta CUDA, u otras a su elección.

Preguntas a responder:

1. Elabore un informe de la actividad realizada en grupo. Se debe utilizar los archivos adjuntos dados por el profesor y verificar el funcionamiento del ejercicio en su equipo, considerando los ambientes:

    a. **Uso de GPU, uso de librería CUDA.**

Ejercicio: Detección de zonas incendio en una imagen satelital

Usar código archivo: <u>CudaEje.cpp</u>
Contexto:

- Imagen en escala de grises (0–255)
- Umbral térmico: **≥ 200**
- Salida binaria (0 = normal, 255 = caliente)
- Tamaño: **2048 × 2048**

```
    // Copiar datos a GPU
    cudaMemcpy(d_img, h_img, size, cudaMemcpyHostToDevice);

    // Eventos CUDA para medir tiempo
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    cudaEventRecord(start);

    int threads = 256;
    int blocks = (WIDTH * HEIGHT + threads - 1) / threads;

    // Lanzamiento del kernel
    detectFire<<<blocks, threads>>>(d_img, d_out);

    cudaEventRecord(stop);
    cudaEventSynchronize(stop);

    float time;
    cudaEventElapsedTime(&time, start, stop);

    printf("Tiempo CUDA: %f ms\n", time);

    // Copiar resultados a CPU
    cudaMemcpy(h_out, d_out, size, cudaMemcpyDeviceToHost);
```

Figura 1. Uso de Cuda.

<u>Para ejecutar digite:</u>
nvcc cudaEje.cpp -02 -o cudaEje
./cudaEje

Se debe adjutar el informe final de la actividad **Perez_Juan_APE3.pdf**

RESULTADOS:

El código fuente *"Cuda.cpp"* tuvo que ser modificado debido a los errores que aparecían durante su implementación en el sistema operativo Ubuntu, como se observa en la Figura 2. Dichos inconvenientes se solucionaron con la inclusión de la librería *<cstdlib>* y cambiando el nombre del archivo a *"Cuda .cu",* pues también existían errores con las variables *blockIdx, blockDim* y *threadIdx* que solo existen si se ejecuta el archivo como CUDA (.cu).

Al compilar con ***nvcc Cuda.cu -o Cuda*** y ejecutar con ***./Cuda*** se observa en la salida el tiempo de ejecución (83.48 milisegundos).

```
luis@ubuntu: ~/Descargas
Cuda.cpp:30:44: error: 'malloc' was not declared in this scope
   30 |     unsigned char *h_img = (unsigned char*)malloc(size);
      |                                            ^~~~~~
Cuda.cpp:3:1: note: 'malloc' is defined in header '<cstdlib>'; did you forget to '#include <cstdlib>'?
    2 | #include <cuda_runtime.h>
  +++ |+#include <cstdlib>
    3 |
Cuda.cpp:35:20: error: 'rand' was not declared in this scope
   35 |         h_img[i] = rand() % 256;
      |                    ^~~~
Cuda.cpp:56:17: error: expected primary-expression before '<' token
   56 |     detectFire<<<blocks, threads>>>(d_img, d_out);
      |                ^
Cuda.cpp:56:35: error: expected primary-expression before '>' token
   56 |     detectFire<<<blocks, threads>>>(d_img, d_out);
      |                                  ^
Cuda.cpp:72:5: error: 'free' was not declared in this scope
   72 |     free(h_img);
      |     ^~~~
Cuda.cpp:72:5: note: 'free' is defined in header '<cstdlib>'; did you forget to '#include <cstdlib>'?
luis@ubuntu:~/Descargas$ nvcc Cuda.cpp -o Cuda
Cuda.cpp: In function 'void detectFire(unsigned char*, unsigned char*)':
Cuda.cpp:13:15: error: 'blockIdx' was not declared in this scope
   13 |     int idx = blockIdx.x * blockDim.x + threadIdx.x;
      |               ^~~~~~~~
Cuda.cpp:13:28: error: 'blockDim' was not declared in this scope
   13 |     int idx = blockIdx.x * blockDim.x + threadIdx.x;
      |                            ^~~~~~~~
Cuda.cpp:13:41: error: 'threadIdx' was not declared in this scope
   13 |     int idx = blockIdx.x * blockDim.x + threadIdx.x;
      |                                         ^~~~~~~~~
Cuda.cpp: In function 'int main()':
Cuda.cpp:57:17: error: expected primary-expression before '<' token
   57 |     detectFire<<<blocks, threads>>>(d_img, d_out);
      |                ^
Cuda.cpp:57:35: error: expected primary-expression before '>' token
   57 |     detectFire<<<blocks, threads>>>(d_img, d_out);
      |                                  ^
luis@ubuntu:~/Descargas$ mv Cuda.cpp Cuda.cu
luis@ubuntu:~/Descargas$ nvcc Cuda.cu -o Cuda
luis@ubuntu:~/Descargas$ nvcc Cuda.cu -o Cuda
luis@ubuntu:~/Descargas$ nvcc Cuda.cu -o Cuda
luis@ubuntu:~/Descargas$ ./Cuda
Tiempo CUDA: 83.489532 ms
luis@ubuntu:~/Descargas$
```

Figura 2. Resultados de compilación y ejecución de código fuente.

En la figura 3 se observa los resultados de la ejecución del comando “top”, que muestra los principales procesos en ejecución, entre ellos, Cuda, que está activo usando que un pequeño porcentaje de CPU, lo que indica que el programa se está ejecutando pero la mayor carga de trabajo probablemente está en la GPU, no en el procesador.

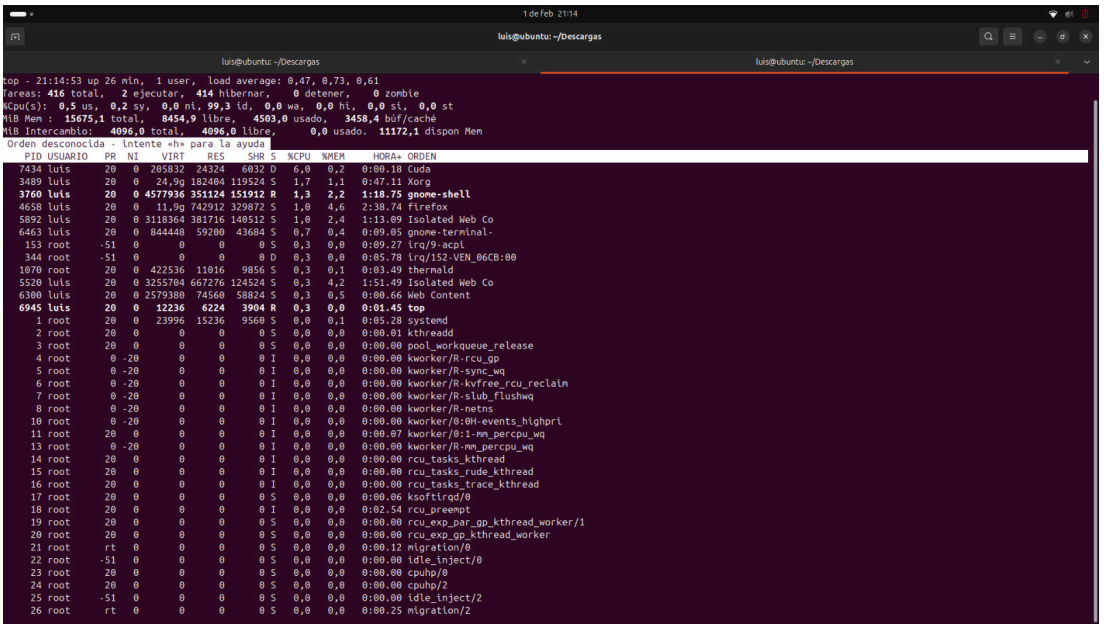

Figura 3. Resultados del comando “top”.

Finalmente en la figura 4 se observan detalles del compilador CUDA (v. 12.0) y la tarjeta gráfica, una RTX 4060, con driver 580.95.05 y soporte para CUDA 13.0.

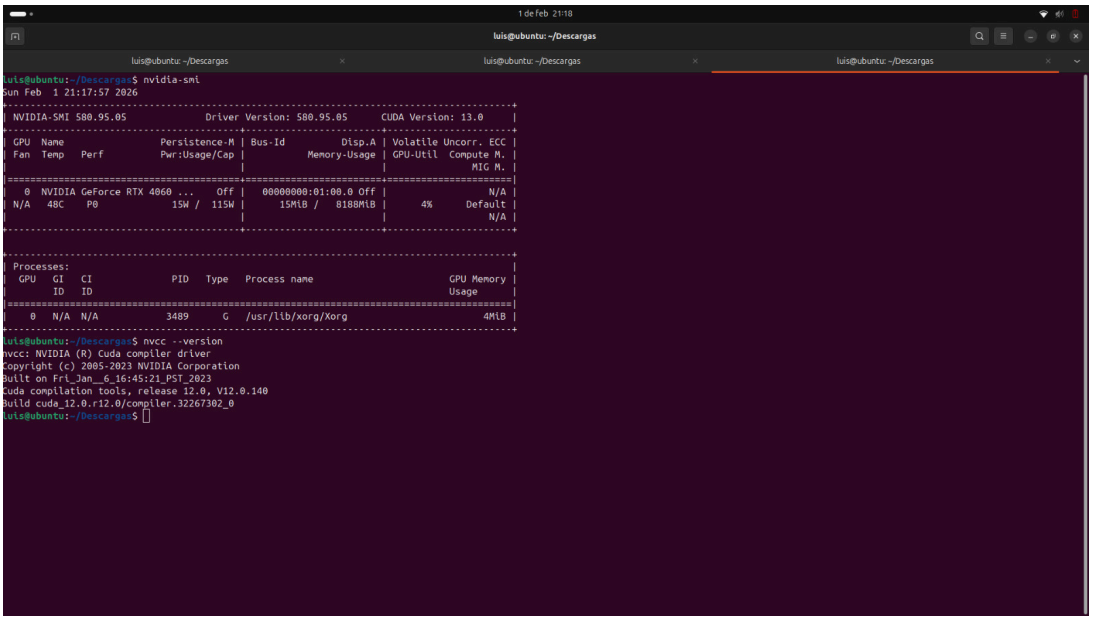

Figura 4. Información de GPU y CUDA

COMPARACIÓN

Con respecto a los resultados del trabajo anterior se observa una notable diferencia en los tiempos de ejecución (0.08 segundos CUDA frente a 0.82 - 0.27 segundos con OpenMP o MPI) ya que, CUDA ejecuta el trabajo de forma masivamente paralela en la GPU, utilizando miles de hilos, por otro lado, OpenMP y MPI solo paralelizan a nivel de CPU, que tiene menor número de núcleos.

A pesar de que CUDA tiene un costo extra al copiar los datos a la GPU, eso se recompensa con un cálculo mucho más rápido por hilo en el procesamiento de píxeles; además, en CPU el rendimiento se estanca a partir de cierto número de núcleos por sobrecarga de sincronización y límites de memoria.

```
C:\Users\Asus\Documents\DocsUNL\Ciclo7\Algoritmos\Unidad 3>g++ deteccion_incendios_openmp.cpp -o deteccion_incendios_openmp -fopenmp -std=c++11

C:\Users\Asus\Documents\DocsUNL\Ciclo7\Algoritmos\Unidad 3>deteccion_incendios_openmp 10000 10000 1
=== OpenMP ===
Resoluci├│n: 10000 x 10000
Hilos: 1
Incendios detectados: 23328013
Tiempo: 0.824 segundos

C:\Users\Asus\Documents\DocsUNL\Ciclo7\Algoritmos\Unidad 3>deteccion_incendios_openmp 10000 10000 4
=== OpenMP ===
Resoluci├│n: 10000 x 10000
Hilos: 4
Incendios detectados: 23328013
Tiempo: 0.286 segundos

C:\Users\Asus\Documents\DocsUNL\Ciclo7\Algoritmos\Unidad 3>deteccion_incendios_openmp 10000 10000 8
=== OpenMP ===
Resoluci├│n: 10000 x 10000
Hilos: 8
Incendios detectados: 23328013
Tiempo: 0.282 segundos

C:\Users\Asus\Documents\DocsUNL\Ciclo7\Algoritmos\Unidad 3>deteccion_incendios_openmp 10000 10000 16
=== OpenMP ===
Resoluci├│n: 10000 x 10000
Hilos: 16
Incendios detectados: 23328013
Tiempo: 0.27 segundos
```

Figura 5. Resultados con OpenMP.

Conclusiones:

- Con el análisis del rendimiento de algoritmos paralelos usando GPU con CUDA se pudo evidenciar mejoras significativas en los tiempos de ejecución en comparación con la ejecución secuencial y paralela a través de OpenMP y MPI, específicamente hubo una mejora del 70% con esta herramienta de NVIDIA, pasando de 270 milisegundos a 80, obteniendo una diferencia considerable de 190 milisegundos.
- El uso de CUDA nos permitió aprovechar de forma más eficiente la capacidad de procesamiento de la GPU, y demostrar que una correcta implementación de algoritmos paralelos mejora el uso de recursos computacionales.

D. RÚBRICA DE EVALUACIÓN	
Informe de trabajo: ● Contenido: pertinente y concreto. ● Estructura y organización: Elementos vinculados y estructurados coherentemente. ● Originalidad y creatividad: trabajo inédito, presentación de nuevas ideas.	2 puntos
Resolución de Preguntas: ● Proceso de resolución de preguntas: con originalidad y creatividad	6 puntos
Conclusiones: ● Redacción ● Originalidad y creatividad: conclusiones inéditas en base a su experiencia y objetivos	2 puntos
Total (Ponderado en Aprendizaje Práctico Experimental)	10 puntos

E. FIRMAS DE RESPONSABILIDAD:	
Estudiante(s):	**Firma**

Gerardo Israel Herrera Campoverde	
Bayron Alejandro Jimenez Cango	
Alexis Gradyn Ludeña Cueva	Alexis Ludeña
Luis Alberto Medina Chamba	
Nayely Cruzcaya Ramirez Herrera	